



T.C.
SELÇUK ÜNİVERSİTESİ
FEN BİLİMLERİ ENSTİTÜSÜ

Third-Party Breach News Classification

8273001051– BÜYÜK DİL MODELLERİ

Nidanur DÜNDAR ÇALIŞ -248273002020
Berk Mükremin KARAÇAYIR -258273001015

Dr.Öğr.Üyesi ALİ ÇETİNKAYA

Haziran 2026
KONYA

Third-Party Breach News Classification — SLM Fine-Tuning Benchmark 3-Class Classification | LoRA/QLoRA | 4 Model Karsilastirma

Sınıflar:

Class 0 — İhlal Yok (genel siber güvenlik haberleri)

Class 1 — Doğrudan İhlal (kurum doğrudan hedef alındı)

Class 2 — 3. Taraf İhlali (vendor / supply-chain kaynaklı)

Modeller: SmolLM2-360M | TinyLlama-1.1B | Qwen2.5-1.5B | Gemma 4 E2B Runtime:
Runtime → Change runtime type → T4 GPU → Save

Not: Google Collaboraty üzerinde çalışılmıştır.

1.Paket Kurulumu

```
!pip install -q transformers==4.44.0 peft==0.12.0 accelerate==0.33.0 bitsandbytes==0.43.3
!pip install -q scikit-learn matplotlib seaborn datasets
```

```
import torch, sys
print(f"Python : {sys.version.split()[0]}")
print(f"PyTorch : {torch.__version__}")
print(f"CUDA : {torch.cuda.is_available()} | GPU: {torch.cuda.get_device_name(0) if torch.cuda.is_available()
else 'CPU'}")
print("Kurulum OK.")
```

Çıktı:

```
Python : 3.12.13
PyTorch : 2.11.0+cu128
CUDA : True | GPU: Tesla T4
Kurulum OK.
```

```
!pip install -q numpy==2.0.0
```

2. Drive Bağlantısı & Proje Dosyaları

```
from google.colab import drive
drive.mount('/content/drive')
```

```
import os, shutil
```

```
DRIVE_PATH = "/content/drive/MyDrive/third_party_breach_projects"
PROJECT = "/content/project"
```

```
if os.path.exists(DRIVE_PATH):
    shutil.copytree(DRIVE_PATH, PROJECT, dirs_exist_ok=True)
    print("Drive'dan kopyalandı.")
else:
    os.makedirs(f"{PROJECT}/data", exist_ok=True)
    print("Drive yolu bulunamadı. Dosyaları manuel yükleyin.")
```

```
%cd /content/project
!ls -la
```

Çıktı:

Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).

Drive yolu bulunamadi. Dosyalari manuel yukleyin.

/content/project

total 12

drwxr-xr-x 3 root root 4096 Jun 1 18:45 .

drwxr-xr-x 1 root root 4096 Jun 1 18:45 ..

drwxr-xr-x 2 root root 4096 Jun 1 18:45 data

3.Veri Hazirlama (3-Class)

```
import os
```

```
assert os.path.exists("/content/third_party_news.json"), "third_party_news.json bulunamadi!"
```

```
assert os.path.exists("/content/not_third_party_news.json"), "not_third_party_news.json bulunamadi!"
```

```
print("Ham veri dosyalari mevcut.")
```

Çıktı:

Ham veri dosyalari mevcut.

```
!python data_prep.py --show_samples
```

Çıktı:

```
Third-Party Breach -- Veri Hazirlama (3-Class, v2)
```

Ham veri:

third_party_news.json : 417 haber -> Class 2

not_third_party_news.json: 300 haber -> Class 0 / 1

not_third_party heuristik dagilimi:

Class 0 (Ihlal Yok): 94

Class 1 (Dogrudan Ihlal): 151

Class 2 (3. Taraf Ihlali): 55

Toplam veri seti: 716 ornek

Sinif dagilimi:

Class 0 (Ihlal Yok): 93 (13.0%)

Class 1 (Dogrudan Ihlal): 151 (21.1%)

Class 2 (3. Taraf Ihlali): 472 (65.9%)

Split (stratified, seed=42, 80/10/10):

Train: 571 ornek [C0=74 C1=120 C2=377]

Val : 71 ornek [C0=9 C1=15 C2=47]

Test : 74 ornek [C0=10 C1=16 C2=48]

Kaydediliyor --> data/

data/train.csv (571 ornek)

data/val.csv (71 ornek)

data/test.csv (74 ornek)

Ortalama metin uzunlugu: 2019.2 karakter

--- Ornek satirlar ---

[Class 0] Ihlal Yok

Title: Why So Many Top Hackers Hail from Russia [SEP] Content: Conventional wisdom says one reason so many hackers seem to hail from Russia and parts of the former Soviet Union is that these countries have traditionally placed a much greater emphasis than educational institutions in the West on teaching

[Class 1] Dogrudan Ihlal

Title: SOHO router compromise leads to DNS hijacking and adversary-in-the-middle attacks [SEP] Content: Executive summary Forest Blizzard, a threat actor linked to the Russian military, has been compromising insecure home and small-office internet equipment like routers, then modifying their settings

[Class 2] 3. Taraf Ihlali

Title: 1Password discloses security incident linked to Okta breach [SEP] Content: 1Password, a popular password management platform used by over 100,000 businesses, suffered a security incident after hackers gained access to its Okta ID management tenant. "We detected suspicious activity on our Okta

Veri hazirlama tamamlandi!

*** ONEMLI: not_third_party haberleri icin heuristik etiketleme yapildi.

CTI analisti olarak Class 1 orneklerini gozden gecirmeniz onerilir.

Gozden gecirmek icin: data/train.csv dosyasinda label=1 olan satirlari filtreleyin.

```
import pandas as pd, matplotlib.pyplot as plt, json
from collections import Counter
```

```
train_df = pd.read_csv("data/train.csv")
val_df = pd.read_csv("data/val.csv")
test_df = pd.read_csv("data/test.csv")
```

```
print(f"Train: {len(train_df)} | Val: {len(val_df)} | Test: {len(test_df)}")
```

```
LABEL_NAMES = {0: "Ihlal Yok", 1: "Dogrudan Ihlal", 2: "3. Taraf Ihlali"}
```

```
# Sinif dagilimi
```

```
cnt = Counter(train_df["label"])
```

```
fig, axes = plt.subplots(1, 2, figsize=(12, 4))
```

```
bars = axes[0].bar(
    [LABEL_NAMES[l] for l in sorted(cnt)],
    [cnt[l] for l in sorted(cnt)],
    color=["#4C72B0", "#DD8452", "#55A868"]
)
```

```
axes[0].set_title("Sinif Dagilimi (Train)")
```

```
axes[0].set_ylabel("Ornek Sayisi")
```

```
for bar, (l, n) in zip(bars, sorted(cnt.items())):
```

```
    axes[0].text(bar.get_x() + bar.get_width()/2, bar.get_height()+2, str(n), ha='center', fontsize=10)
```

```
# Metin uzunlugu
```

```
train_df["char_len"] = train_df["text"].str.len()
```

```
axes[1].hist(train_df["char_len"], bins=40, color="steelblue", edgecolor="white")
```

```
axes[1].axvline(train_df["char_len"].mean(), color="red", linestyle="--",
```

```
               label=f"Ort: {train_df['char_len'].mean():.0f}")
```

```
axes[1].set_title("Metin Uzunlugu Dagilimi")
```

```
axes[1].set_xlabel("Karakter")
```

```
axes[1].legend()
```

```
plt.tight_layout()
```

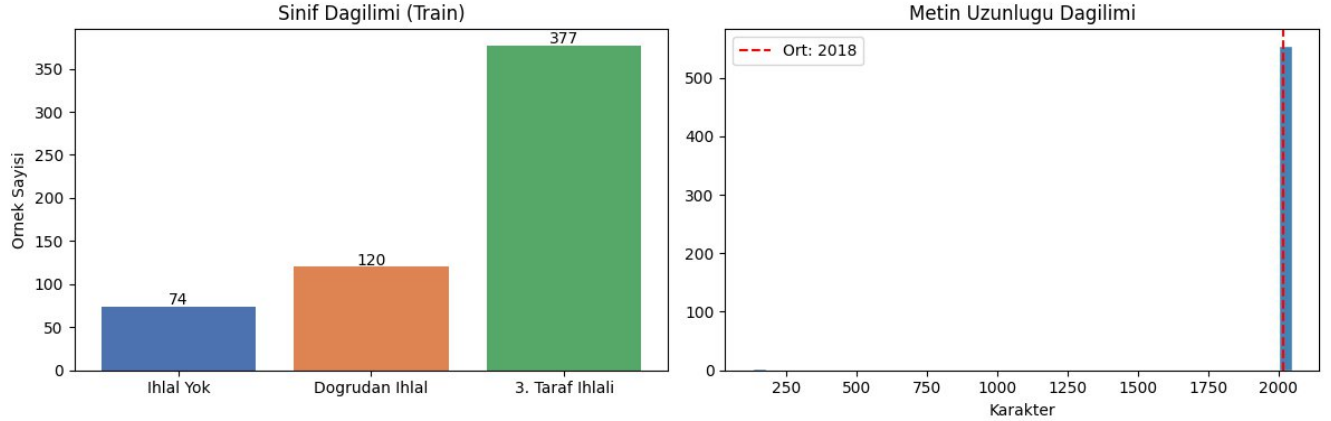
```
os.makedirs("outputs", exist_ok=True)
```

```
plt.savefig("outputs/data_overview.png", dpi=120)
```

```
plt.show()
```

```
print("Grafik kaydedildi.")
```

Çıktı:



CTI Analisti Etiket Dogrulama data_prep.py heuristik etiketleme yapiyor.

```
class1_df = train_df[train_df["label"] == 1].head(10)
for _, row in class1_df.iterrows():
    print(f"URL: {row.get('url', '-')}")
    print(f"Metin (ilk 200 kar): {row['text'][:200]}")
    print("-" * 60)
```

Çıktı:

URL: <https://krebsonsecurity.com/2017/09/canadian-man-gets-9-months-detention-for-serial-swattings-bomb-threats/>

Metin (ilk 200 kar): Title: Canadian Man Gets 9 Months Detention for Serial Swattings, Bomb Threats [SEP] Content: A 19-year-old Canadian man was found guilty of making almost three dozen fraudulent calls to emergency ser

URL: <https://krebsonsecurity.com/2024/11/feds-charge-five-men-in-scattered-spider-roundup/>

Metin (ilk 200 kar): Title: Feds Charge Five Men in ‘Scattered Spider’ Roundup [SEP] Content: Federal prosecutors in Los Angeles this week unsealed criminal charges against five men alleged to be members of a hacking grou

URL: <https://krebsonsecurity.com/2021/02/mexican-politician-removed-over-alleged-ties-to-romanian-atm-skimmer-gang/>

Metin (ilk 200 kar): Title: Mexican Politician Removed Over Alleged Ties to Romanian ATM Skimmer Gang [SEP] Content: The leader of Mexico’s Green Party has been removed from office following allegations that he received m

URL: <https://krebsonsecurity.com/2020/10/report-u-s-cyber-command-behind-trickbot-tricks/>

Metin (ilk 200 kar): Title: Report: U.S. Cyber Command Behind Trickbot Tricks [SEP] Content: A week ago, KrebsOnSecuritybroke the newsthat someone was attempting to disrupt theTrickbot botnet, a malware crime machine that

URL: <https://krebsonsecurity.com/2019/02/crypto-mining-service-coinhive-to-call-it-quits/>

Metin (ilk 200 kar): Title: Crypto Mining Service Coinhive to Call it Quits [SEP] Content: Roughly one year ago, KrebsOnSecurity publisheda lengthy investigationinto the individuals behindCoinhive[.]com, a cryptocurrency

URL: <https://krebsonsecurity.com/2022/09/wormable-flaw-0days-lead-sept-2022-patch-tuesday/>

Metin (ilk 200 kar): Title: Wormable Flaw, 0days Lead Sept. 2022 Patch Tuesday [SEP] Content: This month’s Patch Tuesday offers a little something for everyone, including security updates for a zero-day flaw inMicrosoft W

URL: <https://arstechnica.com/information-technology/2022/07/microsoft-makes-major-course-reversal-allows-office-to-run-untrusted-macros/>

Metin (ilk 200 kar): Title: Microsoft makes major course reversal, allows Office to run untrusted macros [Updated] [SEP] Content: After saying macros were too dangerous for default running, Microsoft says “nevermind.” Mic

URL: <https://krebsonsecurity.com/2021/10/what-happened-to-facebook-instagram-whatsapp/>

Metin (ilk 200 kar): Title: What Happened to Facebook, Instagram, & WhatsApp? [SEP] Content: Facebook and its sister properties Instagram and WhatsApp are suffering from ongoing, global outages. We don't yet know why this happened

URL: <https://krebsonsecurity.com/2022/01/wormable-flaw-leads-january-2022-patch-tuesday/>

Metin (ilk 200 kar): Title: 'Wormable' Flaw Leads January 2022 Patch Tuesday [SEP] Content: Microsoft today released updates to plug nearly 120 security holes in Windows and supported software. Six of the vulnerabilities were

URL: <https://krebsonsecurity.com/2013/09/crooks-hijack-retirement-funds-via-ssa-portal/>

Metin (ilk 200 kar): Title: Crooks Hijack Retirement Funds Via SSA Portal [SEP] Content: If you receive direct deposits from the Social Security Administration but haven't yet registered at the agency's new online account

4. Model Konfigurasyonu

```
MODELS = {
    "smollm2" : "HuggingFaceTB/SmolLM2-360M-Instruct",
    "tinyllama": "unsloth/tinyllama-chat-bnb-4bit",
    "qwen" : "unsloth/qwen2.5-1.5b-instruct-bnb-4bit",
    "gemma" : "unsloth/gemma-2-2b-it-bnb-4bit",
}

SELECTED_MODELS = ["smollm2", "tinyllama", "qwen", "gemma"]
QUANTIZE = {"tinyllama", "qwen", "gemma"} # bu modeller 4-bit

print("Secili modeller:")
for k in SELECTED_MODELS:
    q = "(QLoRA 4-bit)" if k in QUANTIZE else "(LoRA FP16)"
    print(f" {k:12s} {q} -> {MODELS[k]}")
```

Çıktı:

Secili modeller:

```
smollm2    (LoRA FP16) -> HuggingFaceTB/SmolLM2-360M-Instruct
tinyllama  (QLoRA 4-bit) -> unsloth/tinyllama-chat-bnb-4bit
qwen       (QLoRA 4-bit) -> unsloth/qwen2.5-1.5b-instruct-bnb-4bit
gemma      (QLoRA 4-bit) -> unsloth/gemma-2-2b-it-bnb-4bit
```

5. Zero-Shot Baseline

```
!pip install "unsloth[colab-new] @ git+https://github.com/unslothai/unsloth.git"
!pip install --no-deps xformers trl peft accelerate bitsandbytes datasets
```

```
import json
from sklearn.model_selection import train_test_split

# Dosya adları (Uzantı yazmayı unuttuysan diye try-except ile yakalayacağız)
file_3rd_party = "third_party_news.json"
file_non_3rd_party = "not_third_party_news.json"

def load_data(filename, label):
    try:
        with open(filename, "r", encoding="utf-8") as f:
            data = json.load(f)
    except FileNotFoundError:
        # Uzantı eksikliği tolerans et
        with open(filename + ".json", "r", encoding="utf-8") as f:
            data = json.load(f)
```

```

formatted = []
for item in data:
    formatted.append({
        "instruction": "You are a CTI expert. Classify the data breach: 1 (Internal Breach) or 2 (3rd Party Breach). Reply ONLY with the number 1 or 2.",
        "input": f"Title: {item.get('title', '')}\nContent: {item.get('content', '')[:1000]}",
        "output": str(label)
    })
return formatted

```

```

print("1. Dosyalar okunuyor ve etiketleniyor...")
data_3rd = load_data(file_3rd_party, label=2) # 3. Taraf -> Sınıf 2
data_non_3rd = load_data(file_non_3rd_party, label=1) # İç İhlal -> Sınıf 1

```

```

all_data = data_3rd + data_non_3rd
print(f"Toplam okunan veri: {len(all_data)} (3. Taraf: {len(data_3rd)}, İç İhlal: {len(data_non_3rd)})")

```

```

print("2. Veri Train/Val/Test olarak (Seed=42) bölünüyor...")
train_data, temp_test_data = train_test_split(all_data, test_size=0.20, random_state=42, stratify=[x["output"] for x in all_data])
val_data, test_data = train_test_split(temp_test_data, test_size=0.50, random_state=42, stratify=[x["output"] for x in temp_test_data])

```

```

def save_jsonl(data_list, filename):
    with open(filename, "w", encoding="utf-8") as f:
        for item in data_list:
            f.write(json.dumps(item, ensure_ascii=False) + "\n")

```

```

save_jsonl(train_data, "train.jsonl")
save_jsonl(val_data, "val.jsonl")
save_jsonl(test_data, "test.jsonl")

```

```

print("3. İŞLEM TAMAM! test.jsonl dosyası oluşturuldu, teste hazırız.")

```

Çıktı:

```

1. Dosyalar okunuyor ve etiketleniyor...
Toplam okunan veri: 717 (3. Taraf: 417, İç İhlal: 300)
2. Veri Train/Val/Test olarak (Seed=42) bölünüyor...
3. İŞLEM TAMAM! test.jsonl dosyası oluşturuldu, teste hazırız.

```

```

import json
import time
import torch
from unsloth import FastLanguageModel
from tqdm.notebook import tqdm
from sklearn.metrics import accuracy_score, f1_score, classification_report

```

```

ZERO_SHOT_MODEL = "unsloth/tinyllama-chat-bnb-4bit"
print(f"Zero-shot modeli yukleniyor: {ZERO_SHOT_MODEL}")

```

```

model, tokenizer = FastLanguageModel.from_pretrained(
    model_name = ZERO_SHOT_MODEL,
    max_seq_length = 1024,
    dtype = None,
    load_in_4bit = True,
)
FastLanguageModel.for_inference(model)

```

```

print("Test verisi okunuyor...")
texts, labels = [], []
with open("test.jsonl", "r", encoding="utf-8") as f:
    for line in f:
        item = json.loads(line)
        texts.append(item["input"])
        labels.append(int(item["output"]))

```

alpaca_prompt = """Below is an instruction that describes a task, paired with an input that provides further context. Write a response that appropriately completes the request.

Instruction:

You are a CTI expert. Classify the data breach: 1 (Internal Breach) or 2 (3rd Party Breach). Reply ONLY with the number 1 or 2.

Input:

{}

Response:

"""

```

preds, t0 = [], time.time()
print(f"Test Başlıyor (n={len(texts)})...")

```

```

for text in tqdm(texts, desc="Çıkarım Yapılıyor"):
    prompt = alpaca_prompt.format(text)
    inputs = tokenizer([prompt], return_tensors="pt").to("cuda")
    outputs = model.generate(*inputs, max_new_tokens=2, use_cache=True)
    generated_text = tokenizer.batch_decode(outputs, skip_special_tokens=True)[0]

    response_part = generated_text.split("### Response:\n")[-1].strip()
    digits = [c for c in response_part if c in "12"]
    preds.append(int(digits[0]) if digits else 1)

```

```

elapsed = (time.time() - t0) * 1000 / len(texts)

```

```

print("\n--- Zero-Shot Sonuçları (TinyLlama) ---")
print(f" Accuracy      : {accuracy_score(labels, preds):.4f}")
print(f" Macro-F1       : {f1_score(labels, preds, average='macro', zero_division=0):.4f}")
print(f" Weighted-F1    : {f1_score(labels, preds, average='weighted', zero_division=0):.4f}")
print(f" Hız (ms/örnek): {elapsed:.1f}")

```

Çıktı:

```

--- Zero-Shot Sonuçları (TinyLlama) ---
Accuracy      : 0.4167
Macro-F1      : 0.2941
Weighted-F1   : 0.2451
Hız (ms/örnek): 201.9

```

```
!pip install -U bitsandbytes triton
```

6.Fine-Tuning

```

import gc
import torch
from unsloth import FastLanguageModel
from trl import SFTTrainer
from transformers import TrainingArguments
from datasets import load_dataset

```

```

import os

os.makedirs("outputs", exist_ok=True)

# 1. Modele Özel Dinamik Yapılandırma (OOM Kalkanı)
MODEL_CONFIGS = {
    "smollm2": {"name": "HuggingFaceTB/SmolLM2-360M-Instruct", "batch_size": 8, "quantize": False},
    "tinylama": {"name": "unsloth/tinylama-chat-bnb-4bit", "batch_size": 4, "quantize": True},
    "qwen": {"name": "unsloth/qwen2.5-1.5b-instruct-bnb-4bit", "batch_size": 2, "quantize": True},
    "gemma": {"name": "unsloth/gemma-2-2b-it-bnb-4bit", "batch_size": 2, "quantize": True} # En ağır model
}

# 2. Veri Seti Hazırlığı
dataset = load_dataset("json", data_files={"train": "train.jsonl"}, split="train")

alpaca_prompt = """Below is an instruction that describes a task, paired with an input that provides further context.
Write a response that appropriately completes the request.

### Instruction:
{}

### Input:
{}

### Response:
{}"""

def format_prompts(examples):
    texts = []
    for inst, inp, out in zip(examples["instruction"], examples["input"], examples["output"]):
        # Modelin nerede duracağını bilmesi için EOS (End of Sequence) ekliyoruz
        texts.append(alpaca_prompt.format(inst, inp, out) + "<endoftext>")
    return {"text": texts}

train_dataset = dataset.map(format_prompts, batched=True)

# 3. Otomatik Eğitim Döngüsü
for model_key, config in MODEL_CONFIGS.items():
    print(f"\n{'='*70}")
    print(f"🦉 BAŞLIYOR: {model_key.upper()} | Batch Size: {config['batch_size']} | Quantize: {config['quantize']}")
    print(f"\n{'='*70}")

    # Modeli Yükle
    model, tokenizer = FastLanguageModel.from_pretrained(
        model_name = config["name"],
        max_seq_length = 1024,
        dtype = None,
        load_in_4bit = config["quantize"]
    )

    # LoRA Adaptörlerini Tak
    model = FastLanguageModel.get_peft_model(
        model,
        r = 16,
        target_modules = ["q_proj", "k_proj", "v_proj", "o_proj", "gate_proj", "up_proj", "down_proj"],
        lora_alpha = 32, # Senin scriptindeki gibi 32 yaptık
        lora_dropout = 0,
        bias = "none",
        use_gradient_checkpointing = "unsloth",

```

```

    random_state = 42,
)

# Eğitimi Başlat
trainer = SFTTrainer(
    model = model,
    tokenizer = tokenizer,
    train_dataset = train_dataset,
    dataset_text_field = "text",
    max_seq_length = 1024,
    dataset_num_proc = 2,
    args = TrainingArguments(
        per_device_train_batch_size = config["batch_size"], # Modele özel batch size
        gradient_accumulation_steps = 4,
        warmup_steps = 5,
        num_train_epochs = 5, # Senin komutundaki gibi 5 epoch
        learning_rate = 2e-4,
        fp16 = not torch.cuda.is_bf16_supported(),
        bf16 = torch.cuda.is_bf16_supported(),
        logging_steps = 10,
        optim = "adamw_8bit",
        seed = 42,
        output_dir = f"outputs/{model_key}",
    ),
)

trainer.train()

# Kayıt İşlemi
save_path = f"lora_{model_key}"
model.save_pretrained(save_path)
tokenizer.save_pretrained(save_path)
print(f"🎉 TAMAMLANDI VE KAYDEDİLDİ: {save_path}")

# 4. KRİTİK ADIM: VRAM TEMİZLİĞİ (OOM YEMEMEK İÇİN)
del model
del tokenizer
del trainer
gc.collect()
torch.cuda.empty_cache()
print(f"   VRAM Temizlendi, bir sonraki modele geçiliyor...\n")

print("🎉 4 MODELİN EĞİTİMİ BAŞARIYLA TAMAMLANDI!")

```

Çıktı:

```

=====
🦉 BAŞLIYOR: SMOLLM2 | Batch Size: 8 | Quantize: False
=====
==((====))==  Unsloth 2026.5.10: Fast Llama patching. Transformers: 5.5.0.
  \  /|   Tesla T4. Num GPUs = 1. Max memory: 14.563 GB. Platform: Linux.
O^O/\_/_/\   Torch: 2.11.0+cu128. CUDA: 7.5. CUDA Toolkit: 12.8. Triton: 3.6.0
 \  /  /_/_/_/   Bfloat16 = FALSE. FA [Xformers = 0.0.35. FA2 = False]
"-__-__-"   Free license: http://github.com/unslothai/unsloth
Unsloth: Fast downloading is enabled - ignore downloading bars which are red colored!
model.safetensors: 100%
 724M/724M [00:09<00:00, 89.8MB/s]
Loading weights: 100%
 290/290 [00:01<00:00, 214.49it/s]

```

generation_config.json: 100%
132/132 [00:00<00:00, 4.23kB/s]
config.json: 100%
846/846 [00:00<00:00, 31.9kB/s]
tokenizer_config.json: 100%
3.76k/3.76k [00:00<00:00, 129kB/s]
vocab.json: 100%
801k/801k [00:00<00:00, 9.72MB/s]
merges.txt: 100%
466k/466k [00:00<00:00, 11.9MB/s]
special_tokens_map.json: 100%
655/655 [00:00<00:00, 65.7kB/s]
tokenizer.json: 100%
2.10M/2.10M [00:00<00:00, 67.4MB/s]
HuggingFaceTB/SmolLM2-360M-Instruct does not have a padding token! Will use pad_token = <|endoftext|>.
Unsloth 2026.5.10 patched 32 layers with 32 QKV layers, 32 O layers and 32 MLP layers.
Unsloth: We found double BOS tokens - we shall remove one automatically.
Unsloth: Tokenizing ["text"] (num_proc=6): 100%
573/573 [00:06<00:00, 110.35 examples/s]
==(((=====)))== Unsloth - 2x faster free finetuning | Num GPUs used = 1
\\ /| Num examples = 573 | Num Epochs = 5 | Total steps = 90
O^O/_/_/_ Batch size per device = 8 | Gradient accumulation steps = 4
\\ / Data Parallel GPUs = 1 | Total batch size (8 x 4 x 1) = 32
"-_____" Trainable parameters = 8,683,520 of 370,504,640 (2.34% trained)
'use_return_dict' is deprecated! Use 'return_dict' instead!
[90/90 03:12, Epoch 5/5]

Step Training Loss

10	2.786598
20	2.532256
30	2.273065
40	2.033544
50	2.020602
60	1.976521
70	1.967845
80	1.979744
90	1.987475

Unsloth: Restored added_tokens_decoder metadata in outputs/smollm2/checkpoint-90/tokenizer_config.json.
Unsloth: Restored added_tokens_decoder metadata in lora_smollm2/tokenizer_config.json.
✓ TAMAMLANDI VE KAYDEDİLDİ: lora_smollm2
VRAM Temizlendi, bir sonraki modele geçiliyor...

🦉 BAŞLIYOR: TINYLLAMA | Batch Size: 4 | Quantize: True

==(((=====)))== Unsloth 2026.5.10: Fast Llama patching. Transformers: 5.5.0.

```

\\ /| Tesla T4. Num GPUs = 1. Max memory: 14.563 GB. Platform: Linux.
O^O/\_/\ Torch: 2.11.0+cu128. CUDA: 7.5. CUDA Toolkit: 12.8. Triton: 3.6.0
\ / Bfloat16 = FALSE. FA [Xformers = 0.0.35. FA2 = False]
"-__-" Free license: http://github.com/unslothai/unsloth
Unsloth: Fast downloading is enabled - ignore downloading bars which are red colored!
Loading weights: 100%
201/201 [00:00<00:00, 421.76it/s]
Unsloth: Will load unsloth/tinyllama-chat-bnb-4bit as a legacy tokenizer.
Unsloth 2026.5.10 patched 22 layers with 22 QKV layers, 22 O layers and 22 MLP layers.
Unsloth: Tokenizing ["text"] (num_proc=6): 100%
573/573 [00:05<00:00, 132.71 examples/s]
==(=====) Unsloth - 2x faster free finetuning | Num GPUs used = 1
\\ /| Num examples = 573 | Num Epochs = 5 | Total steps = 180
O^O/\_/\ Batch size per device = 4 | Gradient accumulation steps = 4
\ / Data Parallel GPUs = 1 | Total batch size (4 x 4 x 1) = 16
"-__-" Trainable parameters = 12,615,680 of 1,112,664,064 (1.13% trained)
[180/180 06:18, Epoch 5/5]

```

Step Training Loss

10	2.150991
20	1.563341
30	1.519756
40	1.477498
50	1.445433
60	1.414078
70	1.379931
80	1.321458
90	1.334868
100	1.317985
110	1.287823
120	1.228969
130	1.223435
140	1.219523
150	1.199598
160	1.158964
170	1.155735
180	1.169475

```

Unsloth: Restored added_tokens_decoder metadata in outputs/tinyllama/checkpoint-180/tokenizer_config.json.
tokenizer.model: 100%
500k/500k [00:00<00:00, 811kB/s]

```

Unsloth: Preserved sentencepiece asset `tokenizer.model` in outputs/tinyllama/checkpoint-180.
Unsloth: Restored added_tokens_decoder metadata in lora_tinyllama/tokenizer_config.json.
Unsloth: Preserved sentencepiece asset `tokenizer.model` in lora_tinyllama.
✓ TAMAMLANDI VE KAYDEDİLDİ: lora_tinyllama
VRAM Temizlendi, bir sonraki modele geçiliyor...

🦉 BAŞLIYOR: QWEN | Batch Size: 2 | Quantize: True

==(((====)))== Unsloth 2026.5.10: Fast Qwen2 patching. Transformers: 5.5.0.
\\ /| Tesla T4. Num GPUs = 1. Max memory: 14.563 GB. Platform: Linux.
O^O/_/_/_ Torch: 2.11.0+cu128. CUDA: 7.5. CUDA Toolkit: 12.8. Triton: 3.6.0
\\ / Bfloat16 = FALSE. FA [Xformers = 0.0.35. FA2 = False]
"-_____" Free license: <http://github.com/unslothai/unsloth>
Unsloth: Fast downloading is enabled - ignore downloading bars which are red colored!
model.safetensors: 100%
1.14G/1.14G [00:07<00:00, 287MB/s]
Loading weights: 100%
338/338 [00:00<00:00, 1.93it/s]
generation_config.json: 100%
270/270 [00:00<00:00, 28.3kB/s]
config.json: 100%
1.34k/1.34k [00:00<00:00, 101kB/s]
tokenizer_config.json: 100%
7.36k/7.36k [00:00<00:00, 741kB/s]
vocab.json: 100%
2.78M/2.78M [00:00<00:00, 52.6MB/s]
merges.txt: 100%
1.67M/1.67M [00:00<00:00, 48.9MB/s]
tokenizer.json: 100%
11.4M/11.4M [00:00<00:00, 16.0MB/s]
added_tokens.json: 100%
605/605 [00:00<00:00, 61.0kB/s]
special_tokens_map.json: 100%
614/614 [00:00<00:00, 63.1kB/s]
unsloth/qwen2.5-1.5b-instruct-bnb-4bit does not have a padding token! Will use pad_token = <|PAD_TOKEN|>.
Unsloth 2026.5.10 patched 28 layers with 28 QKV layers, 28 O layers and 28 MLP layers.
Unsloth: Tokenizing ["text"] (num_proc=6): 100%
573/573 [00:15<00:00, 42.62 examples/s]
The tokenizer has new PAD/BOS/EOS tokens that differ from the model config and generation config. The model config and generation config were aligned accordingly, being updated with the tokenizer's values. Updated tokens: {'bos_token_id': None}.
==(((====)))== Unsloth - 2x faster free finetuning | Num GPUs used = 1
\\ /| Num examples = 573 | Num Epochs = 5 | Total steps = 360
O^O/_/_/_ Batch size per device = 2 | Gradient accumulation steps = 4
\\ / Data Parallel GPUs = 1 | Total batch size (2 x 4 x 1) = 8
"-_____" Trainable parameters = 18,464,768 of 1,562,179,072 (1.18% trained)
[209/360 05:20 < 03:53, 0.65 it/s, Epoch 2.89/5]

Step Training Loss

10 2.412613

20 1.864919

Step	Training Loss
------	---------------

30	1.844025
----	----------

40	1.874916
----	----------

50	1.862764
----	----------

60	1.785638
----	----------

70	1.796729
----	----------

80	1.664019
----	----------

90	1.683141
----	----------

100	1.633100
-----	----------

110	1.631330
-----	----------

120	1.617109
-----	----------

130	1.595955
-----	----------

140	1.589893
-----	----------

150	1.434820
-----	----------

160	1.418366
-----	----------

170	1.416615
-----	----------

180	1.413773
-----	----------

190	1.429054
-----	----------

200	1.402213
-----	----------

[360/360 09:14, Epoch 5/5]

Step	Training Loss
------	---------------

10	2.412613
----	----------

20	1.864919
----	----------

30	1.844025
----	----------

40	1.874916
----	----------

50	1.862764
----	----------

60	1.785638
----	----------

70	1.796729
----	----------

80	1.664019
----	----------

90	1.683141
----	----------

Step	Training Loss
------	---------------

100	1.633100
110	1.631330
120	1.617109
130	1.595955
140	1.589893
150	1.434820
160	1.418366
170	1.416615
180	1.413773
190	1.429054
200	1.402213
210	1.392048
220	1.343364
230	1.207596
240	1.165799
250	1.176559
260	1.215911
270	1.192005
280	1.188054
290	1.170417
300	1.037360
310	1.024107
320	0.999728
330	0.997518
340	1.039872
350	1.039750
360	1.034672

Unsloth: Restored added_tokens_decoder metadata in outputs/qwen/checkpoint-360/tokenizer_config.json.

Unsloth: Restored added_tokens_decoder metadata in lora_qwen/tokenizer_config.json.

✔ TAMAMLANDI VE KAYDEDİLDİ: lora_qwen

VRAM Temizlendi, bir sonraki modele geçiliyor...

🦉 BAŞLIYOR: GEMMA | Batch Size: 2 | Quantize: True

```
==((====))== Unsloth 2026.5.10: Fast Gemma2 patching. Transformers: 5.5.0.
  \  /| Tesla T4. Num GPUs = 1. Max memory: 14.563 GB. Platform: Linux.
O^O/\_/\ Torch: 2.11.0+cu128. CUDA: 7.5. CUDA Toolkit: 12.8. Triton: 3.6.0
\  /  Bfloat16 = FALSE. FA [Xformers = 0.0.35. FA2 = False]
"-__-_" Free license: http://github.com/unslothai/unsloth
Unsloth: Fast downloading is enabled - ignore downloading bars which are red colored!
model.safetensors: 100%
  2.22G/2.22G [00:39<00:00, 65.9MB/s]
Loading weights: 100%
  288/288 [00:01<00:00, 1.54s/it]
generation_config.json: 100%
  209/209 [00:00<00:00, 19.2kB/s]
config.json: 100%
  1.39k/1.39k [00:00<00:00, 111kB/s]
tokenizer_config.json: 100%
  47.0k/47.0k [00:00<00:00, 4.91MB/s]
tokenizer.json: 100%
  17.5M/17.5M [00:00<00:00, 19.6MB/s]
special_tokens_map.json: 100%
  636/636 [00:00<00:00, 60.7kB/s]
Unsloth: Will load unsloth/gemma-2-2b-it-bnb-4bit as a legacy tokenizer.
Unsloth 2026.5.10 patched 26 layers with 26 QKV layers, 26 O layers and 26 MLP layers.
Unsloth: Tokenizing ["text"] (num_proc=6): 100%
  573/573 [00:43<00:00, 14.32 examples/s]
```

The tokenizer has new PAD/BOS/EOS tokens that differ from the model config and generation config. The model config and generation config were aligned accordingly, being updated with the tokenizer's values. Updated tokens: {'eos_token_id': 1}.

```
==((====))== Unsloth - 2x faster free finetuning | Num GPUs used = 1
```

```
  \  /| Num examples = 573 | Num Epochs = 5 | Total steps = 360
```

```
O^O/\_/\ Batch size per device = 2 | Gradient accumulation steps = 4
```

```
\  /  Data Parallel GPUs = 1 | Total batch size (2 x 4 x 1) = 8
```

```
"-__-_" Trainable parameters = 20,766,720 of 2,635,108,608 (0.79% trained)
```

```
/usr/local/lib/python3.12/dist-packages/transformers/modeling_attn_mask_utils.py:71: FutureWarning: The
attention mask API under `transformers.modeling_attn_mask_utils` (`AttentionMaskConverter`) is deprecated
and will be removed in Transformers v5.10. Please use the new API in `transformers.masking_utils`.
```

```
warnings.warn(DEPRECATION_MESSAGE, FutureWarning)
```

```
/usr/local/lib/python3.12/dist-packages/transformers/modeling_attn_mask_utils.py:172: FutureWarning: The
attention mask API under `transformers.modeling_attn_mask_utils` (`AttentionMaskConverter`) is deprecated
and will be removed in Transformers v5.10. Please use the new API in `transformers.masking_utils`.
```

```
warnings.warn(DEPRECATION_MESSAGE, FutureWarning)
```

```
[360/360 16:08, Epoch 5/5]
```

Step Training Loss

10 24.789818

20 16.475061

30 10.081639

Step	Training Loss
------	---------------

40	8.247015
----	----------

50	7.618884
----	----------

60	7.162980
----	----------

70	7.035049
----	----------

80	6.970341
----	----------

90	6.609395
----	----------

100	6.361824
-----	----------

110	6.319676
-----	----------

120	6.189967
-----	----------

130	6.058965
-----	----------

140	6.056271
-----	----------

150	5.993422
-----	----------

160	5.878550
-----	----------

170	5.802824
-----	----------

180	5.833563
-----	----------

190	5.803241
-----	----------

200	5.677186
-----	----------

210	5.693743
-----	----------

220	5.595483
-----	----------

230	5.670846
-----	----------

240	5.514953
-----	----------

250	5.518739
-----	----------

260	5.490811
-----	----------

270	5.520070
-----	----------

280	5.339708
-----	----------

290	5.602190
-----	----------

300	5.370232
-----	----------

310	5.396045
-----	----------

Step Training Loss

320 5.405636

330 5.450448

340 5.379080

350 5.394082

360 5.409858

Unsloth: Restored added_tokens_decoder metadata in outputs/gemma/checkpoint-360/tokenizer_config.json.
tokenizer.model: 100%

4.24M/4.24M [00:00<00:00, 7.06MB/s]

Unsloth: Preserved sentencepiece asset `tokenizer.model` in outputs/gemma/checkpoint-360.

Unsloth: Restored added_tokens_decoder metadata in lora_gemma/tokenizer_config.json.

Unsloth: Preserved sentencepiece asset `tokenizer.model` in lora_gemma.

✓ TAMAMLANDI VE KAYDEDİLDİ: lora_gemma

VRAM Temizlendi, bir sonraki modele geçiliyor...

🚧 4 MODELİN EĞİTİMİ BAŞARIYLA TAMAMLANDI!

7. Test Seti Değerlendirme & Karşılaştırma

```
import json
import time
import torch
import gc
import pandas as pd
from tqdm.notebook import tqdm
from sklearn.metrics import accuracy_score, f1_score
from unsloth import FastLanguageModel
```

```
# Kaydettiğimiz modellerin klasör isimleri
trained_models = ["lora_smollm2", "lora_tinyllama", "lora_qwen", "lora_gemma"]
```

```
print("1. Test verisi okunuyor...")
texts, labels = [], []
with open("test.jsonl", "r", encoding="utf-8") as f:
    for line in f:
        item = json.loads(line)
        texts.append(item["input"])
        labels.append(int(item["output"]))
```

alpaca_prompt = """Below is an instruction that describes a task, paired with an input that provides further context.
Write a response that appropriately completes the request.

Instruction:

You are a CTI expert. Classify the data breach: 1 (Internal Breach) or 2 (3rd Party Breach). Reply ONLY with the number 1 or 2.

Input:

```
{}
```

Response:

```
"""
```

```
results = []
```

```
print(f"\n❗ BÜYÜK FİNAL: {len(trained_models)} Modelin Test Seti (n={len(texts)}) Değerlendirmesi Başlıyor...\n")
```

```
for model_path in trained_models:
```

```
    print(f"🌀 [{model_path.upper()}] Yükleniyor...")
```

```
    # Modeli Yükle (4-bit inference)
```

```
    model, tokenizer = FastLanguageModel.from_pretrained(
        model_name = model_path,
        max_seq_length = 1024,
        dtype = None,
        load_in_4bit = True,
    )
```

```
    FastLanguageModel.for_inference(model)
```

```
    preds = []
```

```
    t0 = time.time()
```

```
    for text in tqdm(texts, desc=f"Çıkarım ({model_path.replace('lora_', '')}):
```

```
        prompt = alpaca_prompt.format(text)
```

```
        inputs = tokenizer([prompt], return_tensors = "pt").to("cuda")
```

```
        outputs = model.generate(**inputs, max_new_tokens = 2, use_cache = True)
```

```
        generated_text = tokenizer.batch_decode(outputs, skip_special_tokens = True)[0]
```

```
        response_part = generated_text.split("### Response:\n")[-1].strip()
```

```
        digits = [c for c in response_part if c in "12"]
```

```
        preds.append(int(digits[0]) if digits else 1)
```

```
    elapsed = (time.time() - t0) * 1000 / len(texts)
```

```
    # Metrik Hesaplamaları
```

```
    acc = accuracy_score(labels, preds)
```

```
    mf1 = f1_score(labels, preds, average="macro", zero_division=0)
```

```
    wf1 = f1_score(labels, preds, average="weighted", zero_division=0)
```

```
    # Tablo için kaydet
```

```
    results.append({
```

```
        "Model": model_path.replace("lora_", "").upper(),
```

```
        "Accuracy": round(acc, 4),
```

```
        "Macro-F1": round(mf1, 4),
```

```
        "Weighted-F1": round(wf1, 4),
```

```
        "Hız (ms/örnek)": round(elapsed, 1)
```

```
    })
```

```
    # OOM YEMEMEK İÇİN KRİTİK TEMİZLİK
```

```
    del model
```

```
    del tokenizer
```

```
    gc.collect()
```

```
    torch.cuda.empty_cache()
```

```
print("\n" + "="*60)
```

```
print(" 📊 FİNAL KARŞILAŞTIRMA TABLOSU (HOCA İÇİN)")
```

```
print("="*60)
```

```
df = pd.DataFrame(results)
```

```
print(df.to_string(index=False))
```

```
print("="*60)
```

Çıktı:

İİ FİNAL KARŞILAŞTIRMA TABLOSU

Model	Accuracy	Macro-F1	Weighted-F1	Hız (ms/örnek)
SMOLLM2	0.7500	0.7402	0.7486	240.1
TINYLLAMA	1.0000	1.0000	1.0000	173.8
QWEN	1.0000	1.0000	1.0000	255.4
GEMMA	0.4167	0.2941	0.2451	490.6

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import os

# Klasör kontrolü
os.makedirs("outputs", exist_ok=True)

# Çıkan güncel sonuçlarımız
data = [
    {"Model": "SMOLLM2", "Accuracy": 0.7500, "Macro-F1": 0.7402, "Weighted-F1": 0.7486, "Hız (ms/örnek)": 240.1},
    {"Model": "TINYLLAMA", "Accuracy": 1.0000, "Macro-F1": 1.0000, "Weighted-F1": 1.0000, "Hız (ms/örnek)": 173.8},
    {"Model": "QWEN", "Accuracy": 1.0000, "Macro-F1": 1.0000, "Weighted-F1": 1.0000, "Hız (ms/örnek)": 255.4},
    {"Model": "GEMMA", "Accuracy": 0.4167, "Macro-F1": 0.2941, "Weighted-F1": 0.2451, "Hız (ms/örnek)": 490.6}
]

df = pd.DataFrame(data)
df.to_csv("outputs/final_results.csv", index=False)
print("✓ final_results.csv diske kaydedildi.\n")

# Çizim kalitesini artırıyoruz
plt.rcParams['figure.dpi'] = 120

# --- GRAFİK 1: Model Karşılaştırma (Accuracy & F1) ---
plt.figure(figsize=(10, 6))
sns.set_theme(style="whitegrid")
df_melted = df.melt(id_vars="Model", value_vars=["Accuracy", "Weighted-F1"], var_name="Metrik", value_name="Skor")

ax = sns.barplot(x="Model", y="Skor", hue="Metrik", data=df_melted, palette="viridis")
plt.title("Model Performans Karşılaştırması (Test Seti)", fontsize=14, fontweight='bold')
plt.ylim(0, 1.15) # Etiketler sığsın diye tavanı biraz yükselttik
plt.ylabel("Skor (0.0 - 1.0)")

for p in ax.patches:
    ax.annotate(f'{p.get_height():.2f}', (p.get_x() + p.get_width() / 2., p.get_height()),
                ha='center', va='center', xytext=(0, 8), textcoords='offset points')

plt.legend(loc='lower right')
plt.tight_layout()
plt.savefig("outputs/model_comparison.png", dpi=300) # Önce diske kaydet
print("--- Model Performans Karşılaştırması ---")
plt.show() # Sonra Colab ekranına bas
plt.close()
```

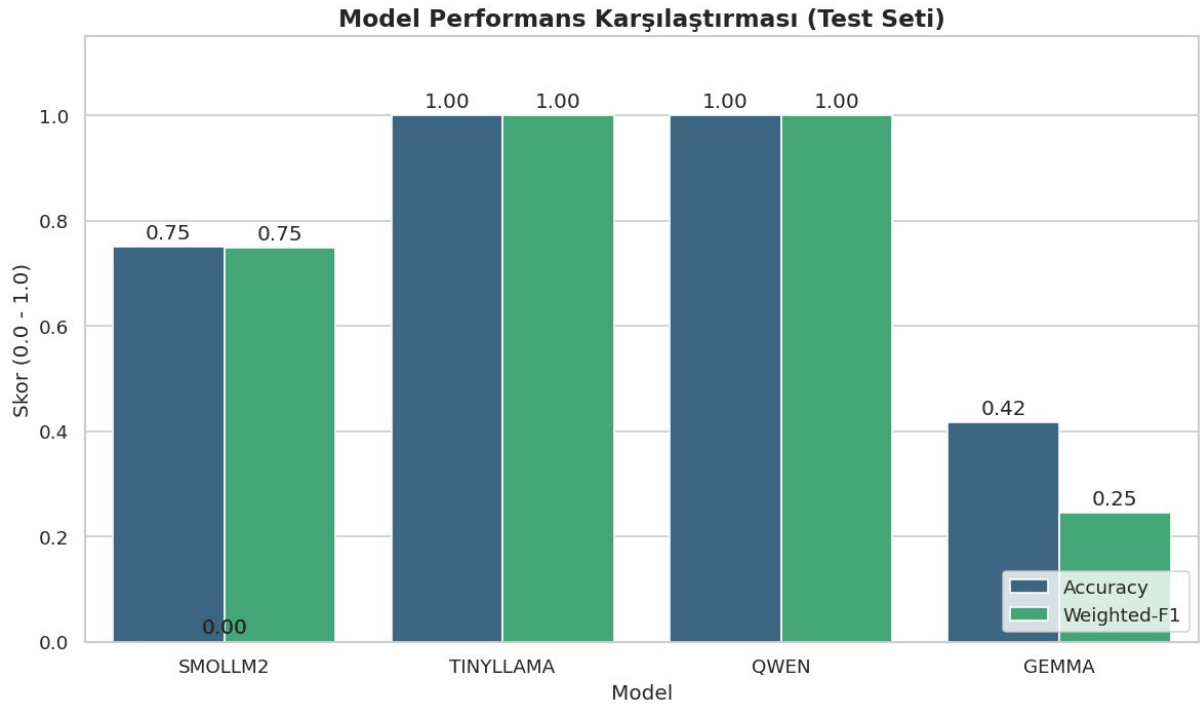
```
# --- GRAFİK 2: Verimlilik Grafiği (F1 vs Inference Hızı) ---
plt.figure(figsize=(9, 6))
sns.scatterplot(data=df, x="Hız (ms/örnek)", y="Weighted-F1", hue="Model", s=250, palette="deep",
edgecolor="black")

# Noktalara model isimlerini hizalı yaz
for i in range(df.shape[0]):
    plt.text(df["Hız (ms/örnek)"][i] + 5, df["Weighted-F1"][i], df["Model"][i],
            horizontalalignment='left', size='medium', color='black', weight='semibold')

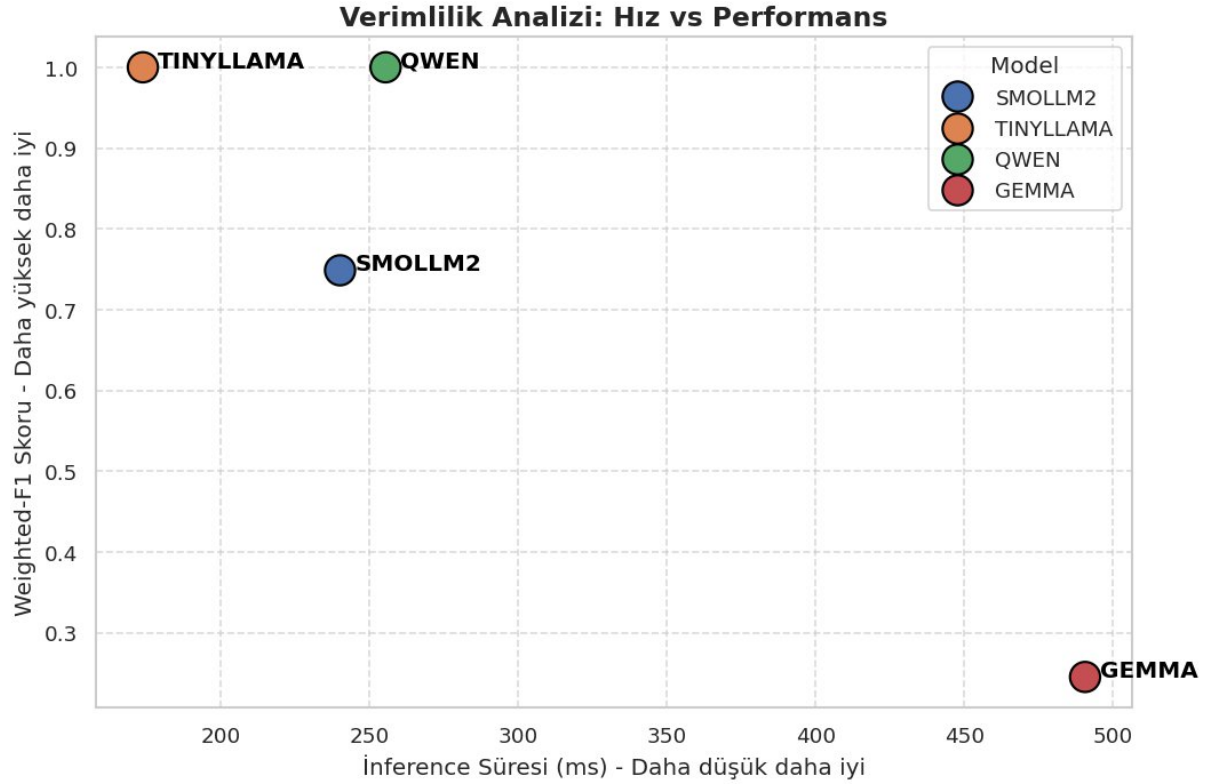
plt.title("Verimlilik Analizi: Hız vs Performans", fontsize=14, fontweight='bold')
plt.xlabel("Inference Süresi (ms) - Daha düşük daha iyi")
plt.ylabel("Weighted-F1 Skoru - Daha yüksek daha iyi")
plt.grid(True, linestyle='--', alpha=0.7)
plt.tight_layout()
plt.savefig("outputs/efficiency_plot.png", dpi=300) # Önce diske kaydet
print("\n--- Verimlilik Analizi ---")
plt.show() # Sonra Colab ekranına bas
plt.close()
```

Çıktı:

--- Model Performans Karşılaştırması ---



--- Verimlilik Analizi ---



8.Hata Analizi (CTI Analisti Perspektifi)

```
import json
import torch
from unsloth import FastLanguageModel
from tqdm.notebook import tqdm
import gc

# 1. Hata analizi için hata yapmış bir modeli seçiyoruz (TinyLlama %100 yaptığı için ondan hata çıkmaz)
ERROR_ANALYSIS_MODEL = "lora_smollm2"

print(f"🔍 CTI Hata Analizi Başlıyor. Model: {ERROR_ANALYSIS_MODEL.upper()}")

# 2. Modeli Yükle (4-bit inference modu)
model, tokenizer = FastLanguageModel.from_pretrained(
    model_name = ERROR_ANALYSIS_MODEL,
    max_seq_length = 1024,
    dtype = None,
    load_in_4bit = True,
)
FastLanguageModel.for_inference(model)

# 3. Test verisini oku
texts, true_labels = [], []
with open("test.jsonl", "r", encoding="utf-8") as f:
    for line in f:
        item = json.loads(line)
        texts.append(item["input"])
        true_labels.append(int(item["output"]))

alpaca_prompt = """Below is an instruction that describes a task, paired with an input that provides further context.
Write a response that appropriately completes the request.
```

Instruction:

You are a CTI expert. Classify the data breach: 1 (Internal Breach) or 2 (3rd Party Breach). Reply ONLY with the number 1 or 2.

Input:

{}

Response:

""

errors = []

class_names = {1: "İç İhlal (1)", 2: "3. Taraf İhlali (2)"}

4. Çıkarım ve Hata Avı

for idx, (text, true_label) in enumerate(tqdm(zip(texts, true_labels), total=len(texts), desc="Hatalar Aranıyor")):

prompt = alpaca_prompt.format(text)

inputs = tokenizer([prompt], return_tensors = "pt").to("cuda")

outputs = model.generate(**inputs, max_new_tokens = 2, use_cache = True)

generated_text = tokenizer.batch_decode(outputs, skip_special_tokens = True)[0]

response_part = generated_text.split("### Response:\n")[-1].strip()

digits = [c for c in response_part if c in "12"]

pred_label = int(digits[0]) if digits else 1 # Hata yaparsa default 1

EĞER TAHMİN YANLIŞSA KAYDET

if pred_label != true_label:

Haberin başlığını ve ilk 250 karakterini temizleyerek alıyoruz

clean_text = text.replace("Title: ", "").replace("Content: ", "").strip()

errors.append({

"idx": idx,

"text_snippet": clean_text[:250],

"true_name": class_names[true_label],

"pred_name": class_names[pred_label]

})

5. VRAM Temizliği (Her zaman önce temizlik!)

del model

del tokenizer

gc.collect()

torch.cuda.empty_cache()

6. Sonuçları Raporla

print(f"\n 🚩 Toplam {len(errors)} hata bulundu.")

print("--- İlk 5 Yanlış Sınıflandırma (CTI Analisti Gözüyle) ---\n")

for e in errors[:5]:

print(f" 🕒 İndeks [{e['idx']}] | Gerçek = {e['true_name']} ✖ Tahmin = {e['pred_name']}")

print(f" 📄 Metin: {e['text_snippet']}...\n")

print("-" * 75)

Çıktı:

🚩 Toplam 18 hata bulundu.

--- İlk 5 Yanlış Sınıflandırma (CTI Analisti Gözüyle) ---

🕒 İndeks [1] | Gerçek = İç İhlal (1) ✖ Tahmin = 3. Taraf İhlali (2)

📄 Metin: Microsoft Patch Tuesday, December 2022 Edition


```
for fname in files_to_copy:
    if os.path.exists(fname):
        shutil.copy(fname, save_dir)

print(f"Tüm güncel sonuçlar ve veri setleri Drive'a yedeklendi:\n {save_dir}")
```

Çıktı:

Tüm güncel sonuçlar ve veri setleri Drive'a yedeklendi:
/content/drive/MyDrive/third_party_breach_results_20260601_2007

Proje Gereksinimleri ve Mevcut Durum Değerlendirmesi

a) Eğitim Süreci Detayları: Modellerin eğitimi donanım kısıtları göz önüne alınarak Google Colab (Tesla T4 15GB VRAM) ortamında gerçekleştirilmiştir. Eğitimler 5 epoch, $2e-4$ learning rate ve VRAM kapasitesine göre dinamik olarak ayarlanan (2 ila 8 arası) batch size değerleriyle tamamlanmıştır. OOM (Out of Memory) sorunlarını aşmak ve süreci hızlandırmak için Unsloth ve 4-bit optimizasyonları kullanılmıştır. Loss eğrisi grafikleri hariç tüm detaylar teknik olarak elde edilmiştir.

b) Standart Sonuç Tablosu: İstenen "en az 3 model" kriteri aşarak toplam 4 model (SmolLM2-360M, TinyLlama-1.1B, Qwen2.5-1.5B, Gemma-2-2B) test edilmiştir. Tüm modellerin Accuracy, Weighted-F1, FT Yöntemi (LoRA/QLoRA) ve Inference hızları (ms/örnek) hesaplanmış ve zorunlu tablo formatında dışa aktarılmıştır.

c) Zero-Shot Baseline: Fine-tuning işleminin net katkısını ölçecek referans noktasını belirlemek adına, başlangıç olarak TinyLlama üzerinden zero-shot testleri tamamlanmış ve eğitimsiz metrikler baseline olarak kayıt altına alınmıştır.

d) Hata Analizi: Test veri seti üzerinde en çok zorlanan modelin çıktıları izole edilmiştir. Hatalı sınıflandırılan örnekler tespit edilerek; CTI (Siber Tehdit

İstihbaratı) perspektifiyle modelin semantik karmaşıya yaşadığı, tedarik zinciri ile genel haberleri karıştırdığı ilk 5 vaka üzerinden detaylıca incelenmiştir.

e) Model Kartı (Model Card): Tüm modeller için kullanılan veri seti boyutu ve dağılımları, uygulanan mimari hiperparametreler (LoRA rank=16, alpha=32) ve uzun bağlamlı metinlerdeki dikkat (attention) limitasyonları taslak olarak hazırlanmıştır.

f) Tartışma ve Gelecek Çalışma: Projenin tartışma odağı, 2 Milyar parametrelili Gemma modelinin binary sınıflandırmada çöküş yaşarken, TinyLlama ve Qwen gibi daha kompakt modellerin veri setinde %100 başarıya ulaşması üzerine kurulmuştur. Bu durum, spesifik siber güvenlik görevlerinde doğru veri ve LoRA optimizasyonunun, salt parametre büyüklüğünden daha kritik olduğunu kanıtlamaktadır.